

Solver-Integrated Adversarial Attacking and Training of Neural Operators

Anonymous submission

Abstract

Neural operators are widely used as fast surrogates for numerical PDE solvers, mapping input functions to solution functions. However, their generalizability and robustness are not yet clearly defined in the operator-learning setting, which differs from traditional adversarial robustness definitions. This paper studies the generalizability and robustness of a learned neural operator from a solver-integrated perspective, addressing the challenge that the output of a learned operator and a numerical solver tends to change in tandem under input perturbation. First, we formalize the definition of generalization and robustness through a model-solver error operator, identifying fixed-input model-solver loss as generalization metric, and norm-bounded adversarial attack loss increase and Jacobian-error function norm as robustness metric. Second, we identify the solver-integrated adversarial attack as appropriate for PDE operator learning and show why model-only or fixed-ground-truth attacks can be insufficient when the solver output also changes with the input. Third, we develop solver-integrated adversarial training methods for neural operators. Experiments on representative PDE benchmarks show that this solver-integrated adversarial training clearly improves both generalizability and robustness. Deeper solver integration yields more effective attacks, more informative samples, and more efficient training than less integrated alternatives. These results provide a general framework for robust operator training and automatic sample selection without heavy manual intervention.

Introduction

Neural operators, including Fourier Neural Operators and DeepONets, learn maps between function spaces and are widely used as fast data-driven surrogates for numerical PDE solvers (Li et al. 2021; Lu et al. 2021). They map input functions to solution functions in one forward pass, making them attractive when many related PDE solution queries are needed. Practical use, however, requires the learned operator to generalize beyond the training dataset and remain close to the solver when the input function is perturbed. Recent neural operator studies have therefore begun to examine adversarial

robustness and robustness-aware training (Adesoji and Chen 2022; Huang et al. 2026; Roy, Chakraborty, and Alam 2026).

Prior work has often not clearly separated the notions of generalization and robustness. Classical theory defines generalization through expected-versus-empirical loss (Bousquet and Elisseeff 2002; Xu and Mannor 2012). Several modern robustness notions are defined by worst-case loss increase under norm-bounded perturbations (Madry et al. 2018), and global Lipschitz constants and local spectral norms are also used to measure robustness (Cissé et al. 2017; Huang et al. 2026). We inherit these ideas and specialize them to the operator learning setting. Generalization is a static definition: the norm of the output of our error operator. Robustness is defined in a dynamic way: how the output of our error operator amplifies under an input perturbation. We further develop Jacobian-error, which is cheaper to compute and closely aligned with adversarial loss increase in the local robustness setting.

Most classical adversarial attacks were developed for classification models, where it is reasonable to assume that a small input perturbation should not change the ground-truth label (Goodfellow, Shlens, and Szegedy 2015; Madry et al. 2018). However, the PDE operator learning problem is not only a regression problem, but a regression problem with an existing numerical solver as a trusted oracle. When the input function is perturbed, the correct solution should change accordingly but stay close to the solver’s output. Therefore, the model is not expected to be robust by itself; rather, the model-solver discrepancy is expected to be robust. We therefore define the error operator as the discrepancy between the learned operator and the solver.

Building on this view, we develop solver-integrated adversarial training. The method can be interpreted as a teacher-student solver distillation framework: the numerical solver acts as the teacher, while the neural operator is the student repeatedly trained on informative inputs discovered from the local model-solver discrepancy, so that the neural operator becomes an increasingly faithful behavioral twin of the solver. In this sense, the solver’s behavior is distilled into the neural operator through automatically generated samples guided by model and solver gradients. This process reduces manual intervention and avoids the bias of hand-selected training inputs. Experiments on representative PDE benchmarks show that the proposed training clearly improves gen-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

eralization and robustness, and that deeper solver integration yields more effective attacks, more informative samples, and more efficient training.

Our main contributions can be summarized as follows:

- In the setting of operator learning, we propose a new robustness definition and metric.
- We identify the most accurate and appropriate target loss for neural-operator adversarial attack, which involves deep solver integration.
- We propose solver-integrated adversarial training for neural operators to improve both generalization and robustness.

Related Work

Traditional adversarial attacks. Most classical adversarial attacks were designed for classification, where the objective is usually to maximize a label loss such as $\ell(f_\theta(x + \delta), y)$ under a perturbation constraint (Goodfellow, Shlens, and Szegedy 2015; Madry et al. 2018). These attacks mainly differ in the optimization procedure, perturbation constraint, sparsity or structure, or available information, while retaining the assumption that the ground-truth label remains fixed (Carlini and Wagner 2017; Papernot et al. 2016; Moosavi-Dezfooli, Fawzi, and Frossard 2016; Brendel, Rauber, and Bethge 2018; Chen, Jordan, and Wainwright 2020; Andriushchenko et al. 2020; Su, Vargas, and Sakurai 2019; Brown et al. 2017; Croce and Hein 2020). Regression attacks and time-series forecasting attacks have also been studied (Nguyen and Raff 2019; Tong et al. 2018; Ribeiro et al. 2023; Mode and Hoque 2020; Liu et al. 2023). Their target losses use a fixed observed label or the model’s own output. They do not recompute the true loss with the true output for the perturbed input inside the attack loop. They acknowledge that the true output should change accordingly with input perturbations, but lack the generating function in that setting. Therefore, they assume that small perturbations do not meaningfully change the observed label. However, in our operator-learning setting, the numerical solver is exactly this generating function, so the true output and the corresponding true loss can be recomputed for each perturbed input inside the loop. Under the framework below, previous attack losses contain relatively little solver integration and fall into the first two target-loss categories, $\mathcal{L}_1$ and $\mathcal{L}_2$.

Neural-operator adversarial attacks. A closer study evaluates adversarial robustness for operator learning and finds that neural-operator error can grow significantly as the perturbation norm increases (Adesoji and Chen 2022). Although solver outputs are used as references, the solver is not truly integrated into the attack loop. In the forward pass, the solver output for an updated perturbed input is approximated by a nearest-neighbor ground-truth output from a precomputed dictionary; in the backward pass, the solver gradient is ignored and only the neural-operator gradient is used. Besides, this study focuses on adversarial robustness evaluation for neural operators, but does not propose a method for improving the model’s adversarial robustness.

PDE-problem adversarial training. WbAR combines adversarial training with PINNs by moving collocation points

toward larger PDE residuals (Shi et al. 2026); StableP-DENet applies adversarial training to neural operators with a physics loss (Huang et al. 2026); and RAMS systematizes this residual-gradient sample movement for PINNs and operator learning (Ouyang et al. 2025). These methods do not use true solver outputs as supervised targets inside the adversarial loop. Instead, they use physics loss as a surrogate loss. The drawbacks are that this physics loss is less informative than direct solver supervision and that, in many time-dependent neural operator settings, the model outputs only one single frame of the final state, so the temporal derivatives required by a physics loss are unavailable. Other operator-learning works use heuristic, active, or generative sampling: neural-operator digital-twin studies use gradient-free sparse attacks and attack-driven active learning with input denoising (Roy et al. 2026; Roy, Chakraborty, and Alam 2026), while adversarial autoencoders, GANO, and turbulent-flow generative models use distributional or generative objectives (Enyeart and Lin 2024; Rahman et al. 2022; Oommen et al. 2025). These sampling strategies can be useful, but they are heuristic or distributional rather than chosen according to the current solver-model discrepancy, and they do not use solver gradients to choose perturbation directions.

Preliminaries

Problem Formulation

Let $\mathcal{X}$ and $\mathcal{Y}$ be Hilbert function spaces equipped with norms $\|\cdot\|_{\mathcal{X},p}$ and $\|\cdot\|_{\mathcal{Y},q}$, respectively, where $p = q = 2$ by default. An input is a function $x \in \mathcal{X}$, such as an initial condition or coefficient field, and the output is a function $y \in \mathcal{Y}$, such as a final condition or solution function. Throughout the paper, the model is the surrogate neural operator $\mathcal{F}_\theta$, while the solver is the oracle ground-truth input-to-output mapping $\mathcal{G}$. We assume these operators are Frechet differentiable at x . Their difference defines the model-solver error operator $\mathcal{E}_\theta$:

$$\mathcal{F}_\theta, \mathcal{G}, \mathcal{E}_\theta : \mathcal{X} \rightarrow \mathcal{Y}, \quad \mathcal{E}_\theta := \mathcal{F}_\theta - \mathcal{G},$$

$$e_\theta[x] := \mathcal{E}_\theta(x) = \mathcal{F}_\theta(x) - \mathcal{G}(x),$$

$$D\mathcal{F}_\theta[x], D\mathcal{G}[x] : \mathcal{X} \rightarrow \mathcal{Y}, \quad D\mathcal{E}_\theta[x] := D\mathcal{F}_\theta[x] - D\mathcal{G}[x].$$

For a perturbation increment $\eta = th \in \mathcal{X}$, let $h \in \mathcal{X}$ be an input-space direction with $\|h\|_{\mathcal{X},p} = 1$. We take $t \rightarrow 0$, and hence $\|\eta\|_{\mathcal{X},p} \rightarrow 0$:

$$\begin{aligned} \mathcal{F}_\theta(x + \eta) &= \mathcal{F}_\theta(x) + D\mathcal{F}_\theta[x](\eta) + o(\|\eta\|_{\mathcal{X},p}), \\ \mathcal{G}(x + \eta) &= \mathcal{G}(x) + D\mathcal{G}[x](\eta) + o(\|\eta\|_{\mathcal{X},p}), \\ e_\theta[x + \eta] &= e_\theta[x] + D\mathcal{E}_\theta[x](\eta) + o(\|\eta\|_{\mathcal{X},p}). \end{aligned}$$

Here $D\mathcal{F}_\theta[x](\eta)$, $D\mathcal{G}[x](\eta)$, and $D\mathcal{E}_\theta[x](\eta)$ are the corresponding first-order output variation functions. We define the radius- ε perturbation ball as the admissible constraint set for the perturbed input in the function space $\mathcal{X}$:

$$\mathcal{B}_{p,\varepsilon}(x) = \{x + \delta : \delta \in \mathcal{X}, \|\delta\|_{\mathcal{X},p} \leq \varepsilon\}.$$

Generalization and Robustness Metrics

1. Generalization. Generalization is the static model-solver discrepancy at a fixed input function:

$$\mathcal{L}_{\text{err}}(x) = \frac{1}{2} \|e_\theta[x]\|_{\mathcal{Y},2}^2 = \frac{1}{2} \|\mathcal{F}_\theta(x) - \mathcal{G}(x)\|_{\mathcal{Y},2}^2.$$

2. Robustness. Robustness asks how the model’s error changes instead of how the model changes when the input function is perturbed. We organize it into three local metrics.

We use h for an infinitesimal direction. Along h ,

$$e_\theta[x + th] = e_\theta[x] + tD\mathcal{E}_\theta[x](h) + o(t), \quad t \rightarrow 0.$$

The first variation of $\mathcal{L}_{\text{err}}$ is

$$D\mathcal{L}_{\text{err}}[x](h) := \left. \frac{d}{dt} \frac{1}{2} \langle e_\theta[x + th], e_\theta[x + th] \rangle_{\mathcal{Y}} \right|_{t=0} \\ = \langle e_\theta[x], D\mathcal{E}_\theta[x](h) \rangle_{\mathcal{Y}} = \langle D\mathcal{E}_\theta[x]^* e_\theta[x], h \rangle_{\mathcal{X}} = \langle g_\theta(x), h \rangle_{\mathcal{X}}$$

where $g_\theta(x) := D\mathcal{E}_\theta[x]^* e_\theta[x] \in \mathcal{X}$, and $D\mathcal{E}_\theta[x]^* : \mathcal{Y} \rightarrow \mathcal{X}$ is the adjoint derivative. After discretization this becomes the vector $J_{\mathcal{E}}(x)^\top e_\theta(x)$. This derivative is the first-order growth rate of the squared model-solver error loss when x is perturbed along h . It is the function-space inner product between the Jacobian-error function $g_\theta(x)$ and the perturbation direction h .

Jacobian norm. The first robustness metric is the induced norm of the derivative of the error operator. In a Lipschitz view, a regional robustness bound uses the largest derivative norm over an input region; at a fixed input x , this reduces to the pointwise induced norm of the Frechet derivative. The Frechet derivative is defined from perturbations $x + th$ as $t \rightarrow 0$; after this infinitesimal scale is factored out, the Jacobian norm maximizes over normalized input-space directions h :

$$\|D\mathcal{E}_\theta[x]\|_{p \rightarrow q} = \sup_{\|h\|_{\mathcal{X},p}=1} \|D\mathcal{E}_\theta[x](h)\|_{\mathcal{Y},q}.$$

For a small perturbation th , the linearized change is $tD\mathcal{E}_\theta[x](h)$. Thus the Jacobian norm gives the first-order growth rate of the nonlinear error operator at x ; the maximizing direction h is the Jacobian norm growth direction. After discretization, with $J = J_{\mathcal{E}}(x)$, one common case is $2 \rightarrow 2 : \sigma_{\max}(J)$ with the leading right singular vector as direction.

This Jacobian norm differs from a model-only local stability diagnostic such as the spectral norm $\|D\mathcal{F}_\theta[x]\|$, which StablePDENet reports for neural operators (Huang et al. 2026). A large $\|D\mathcal{F}_\theta[x]\|$ does not by itself imply poor solver-consistent robustness, because the true solver $\mathcal{G}$ can also have non-negligible input-output sensitivity; both outputs are expected to change when the input changes. The desired local property is that the model and solver change similarly, $D\mathcal{F}_\theta[x] \approx D\mathcal{G}[x]$, so the error derivative $D\mathcal{E}_\theta[x]$ is small.

Adversarial attack loss increase. For a fixed input function x , the attack optimizes over the perturbation δ ; the model, solver, and clean input are fixed. This metric directly measures the worst loss (error’s norm) increase over a norm-bounded perturbation ball, rather than a local error change’s norm. For one perturbation, define

$$\Delta\mathcal{L}_{\text{err}}(\delta) := \mathcal{L}_{\text{err}}(x + \delta) - \mathcal{L}_{\text{err}}(x).$$

Since the clean error $\|e_\theta[x]\|_{\mathcal{Y}}^2$ is constant in δ , maximizing $\|e_\theta[x + \delta]\|_{\mathcal{Y}}^2$ and maximizing $\Delta\mathcal{L}_{\text{err}}(\delta)$ give the same perturbation. We choose a budget-feasible δ that maximizes this loss increase; the chosen δ is the finite-budget adversarial perturbation direction.

Jacobian-error. The third robustness quantity is Jacobian-error, defined by the input-space function $g_\theta(x) = D\mathcal{E}_\theta[x]^* e_\theta[x]$. It combines the derivative of the error operator with the current error field. Its scalar growth value is $\|g_\theta(x)\|_{\mathcal{X}}$, and its growth direction is $g_\theta(x)/\|g_\theta(x)\|_{\mathcal{X}}$ when $g_\theta(x) \neq 0$.

Comparison of Jacobian-error and finite-budget attack.

In the small-budget limit, the finite-budget attack reduces to Jacobian-error. Write $\delta = \varepsilon h$ with $\|h\|_{\mathcal{X}} \leq 1$. As $\varepsilon \rightarrow 0$, the first-order expansion is uniform over $\|h\|_{\mathcal{X}} \leq 1$, so

$$\Delta\mathcal{L}_{\text{err}}^*(\varepsilon) = \max_{\|h\|_{\mathcal{X}} \leq 1} \Delta\mathcal{L}_{\text{err}}(\varepsilon h) \approx \varepsilon \max_{\|h\|_{\mathcal{X}} \leq 1} \langle g_\theta(x), h \rangle_{\mathcal{X}} \\ = \varepsilon \|g_\theta(x)\|_{\mathcal{X}} \quad (h^* = g_\theta(x)/\|g_\theta(x)\|_{\mathcal{X}}).$$

Thus, in the small-budget limit, the finite-budget adversarial attack has the same first-order growth value and growth direction as Jacobian-error.

Comparison of Jacobian-error and Jacobian norm.

Jacobian-error differs from the Jacobian norm because the two quantities optimize different local objectives. Jacobian-error measures the error-aligned first-order growth of the final squared error:

$$\frac{1}{2} \|e_\theta[x + h]\|_{\mathcal{Y}}^2 - \frac{1}{2} \|e_\theta[x]\|_{\mathcal{Y}}^2 \approx \langle e_\theta[x], D\mathcal{E}_\theta[x](h) \rangle_{\mathcal{Y}}$$

This first-order term depends on both the current error $e_\theta[x]$ and the local Jacobian through $D\mathcal{E}_\theta[x](h)$. The Jacobian norm measures the largest local displacement of the error field:

$$\|e_\theta[x + h] - e_\theta[x]\|_{\mathcal{Y}} \approx \|D\mathcal{E}_\theta[x](h)\|_{\mathcal{Y}}$$

Thus, the Jacobian norm maximizes the local error displacement norm, not the error norm (loss) increase. Consequently, in this setting the Jacobian norm is not the most suitable robustness proxy; the Jacobian-error norm is more directly aligned with finite adversarial attack loss increase. For the $2 \rightarrow 2$ norm case ($p = q = 2$), the Jacobian-norm direction is the leading right singular direction of $D\mathcal{E}_\theta[x]$, governed by $D\mathcal{E}_\theta[x]^* D\mathcal{E}_\theta[x]$. Empirically, Jacobian-error tracks finite adversarial attack loss increase more closely than Jacobian norm for small ε . It is also less computationally intensive: for a discretized $J_{\mathcal{E}} \in \mathbb{R}^{m \times n}$ and error vector $e_\theta \in \mathbb{R}^m$, computing $J_{\mathcal{E}}^\top e_\theta$ and its norm costs $O(mn)$, whereas computing the Jacobian norm through the leading singular value/vector costs $O(mn \min\{m, n\})$ for an exact dense SVD or $O(kmn)$ for k iterative matrix-vector passes.

Proposed Framework

Solver-Integrated Adversarial Attack

The attack optimizes the chosen loss over the finite perturbation δ , with x fixed. For iterative variants, $x_k = x + \delta_k$ is the perturbed input at step k , and $\text{sg}(\cdot)$ denotes stop-gradient. We use three primary losses and several implementation vari-

ants:

$$\begin{aligned}
\mathcal{L}_1(\delta) &= \|\mathcal{F}_\theta(x + \delta) - \mathcal{F}_\theta(x)\|_{\mathcal{Y},q}, \\
\mathcal{L}_2(\delta) &= \|\mathcal{F}_\theta(x + \delta) - \mathcal{G}(x)\|_{\mathcal{Y},q}, \\
\mathcal{L}_3(\delta) &= \|\mathcal{F}_\theta(x + \delta) - \mathcal{G}(x + \delta)\|_{\mathcal{Y},q} = \|e_\theta[x + \delta]\|_{\mathcal{Y},q}, \\
\mathcal{L}_2^{\text{fixed}}(\delta_k) &= \|\mathcal{F}_\theta(x_k) - \mathcal{G}(x)\|_{\mathcal{Y},q}, \\
\mathcal{L}_3^{\text{full}}(\delta_k) &= \|\mathcal{F}_\theta(x_k) - \mathcal{G}(x_k)\|_{\mathcal{Y},q}, \\
\mathcal{L}_{2,N}^{\text{dict}}(\delta_k) &= \|\mathcal{F}_\theta(x_k) - y_{\text{dict},N}(x_k)\|_{\mathcal{Y},q}, \\
\mathcal{L}_3^{\text{sg}}(\delta_k) &= \|\mathcal{F}_\theta(x_k) - \text{sg}(\mathcal{G}(x_k))\|_{\mathcal{Y},q}, \\
\mathcal{L}_{\text{phys}}(\delta_k) &= \lambda_{\text{PDE}}\|R_{\text{PDE}}(\mathcal{F}_\theta(x_k))\|^2 \\
&\quad + \lambda_{\text{BC}}\|R_{\text{BC}}(\mathcal{F}_\theta(x_k))\|^2 + \lambda_{\text{IC}}\|R_{\text{IC}}(x_k)\|^2.
\end{aligned}$$

For the squared-loss version, the stop-gradient and full $\mathcal{L}_3$ variants have different backpropagated first-order directions:

$$\begin{aligned}
\nabla_{\delta_k} \frac{1}{2} \mathcal{L}_3^{\text{sg}}(\delta_k)^2 &= D\mathcal{F}_\theta[x_k]^*(\mathcal{F}_\theta(x_k) - \mathcal{G}(x_k)), \\
\nabla_{\delta_k} \frac{1}{2} \mathcal{L}_3^{\text{full}}(\delta_k)^2 &= (D\mathcal{F}_\theta[x_k] - D\mathcal{G}[x_k])^*(\mathcal{F}_\theta(x_k) - \mathcal{G}(x_k)).
\end{aligned}$$

Model-output-change objectives, including some regression attacks and regularizers that compare perturbed and clean predictions, follow $\mathcal{L}_1$ (Nguyen and Raff 2019; Mode and Hoque 2020; Liu et al. 2023). Fixed-label classification, fixed-response regression, specified-target regression/forecasting, and neural-operator tests with held or pre-computed solver targets follow $\mathcal{L}_2$ (Goodfellow, Shlens, and Szegedy 2015; Madry et al. 2018; Tong et al. 2018; Ribeiro et al. 2023; Liu et al. 2023; Adesoji and Chen 2022). We are not aware of prior attack losses that optimize the full $\mathcal{L}_3$.

$\mathcal{L}_{2,N}^{\text{dict}}$ is an $\mathcal{L}_2$ -like variant: it replaces the online solver output by the precomputed solver output attached to the nearest dictionary input, rather than calling the solver during the attack. This is the structure used by dictionary-based FNO robustness evaluation (Adesoji and Chen 2022). The stop-gradient variant uses the solver output in the forward target but blocks the solver branch in backpropagation, whereas $\mathcal{L}_3^{\text{full}}$ differentiates through the model–solver difference as shown above. Physics-residual attacks such as WbAR, StablePDENet, and RAMS instead use $\mathcal{L}_{\text{phys}}$: they do not compare with solver outputs, but replace solver supervision by PDE, boundary, and initial-condition residuals (Shi et al. 2026; Huang et al. 2026; Ouyang et al. 2025). Overall, these attack variants increase solver integration in the order $\mathcal{L}_{\text{phys}}, \mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_{2,N}^{\text{dict}}, \mathcal{L}_3^{\text{sg}}$, and $\mathcal{L}_3^{\text{full}}$. Our experiments below show that deeper solver integration generally yields stronger attacks and more effective adversarial training.

Optimization Methods for Solver-Integrated Attacks

We propose four optimizer variants, and many existing first-order attack optimizers can be viewed within this framework. For a selected attack loss $\mathcal{L}(\delta)$, let $g_k = \nabla_\delta \mathcal{L}(\delta_k)$ denote the input-space gradient at attack step k . Raw updates use g_k directly. The steepest direction is

$$s_k \in \arg \max_{\|v\|_{\mathcal{X},p} \leq 1} \langle g_k, v \rangle.$$

Raw add is standard PGD, and steepest add is ℓ_p -steepest PGD. Add-style methods accumulate directions and project

	Add	Replace
Raw	$\delta_{k+1} = \Pi_{\mathcal{B}_\varepsilon}(\delta_k + \alpha g_k)$	$\delta_{k+1} = \varepsilon g_k / \ g_k\ _{\mathcal{X},p}$
Steepest	$\delta_{k+1} = \Pi_{\mathcal{B}_\varepsilon}(\delta_k + \alpha s_k)$	$\delta_{k+1} = \varepsilon s_k$

back to the perturbation ball, so early steps may move toward the boundary. Replace-style methods discard the previous perturbation and directly place the new perturbation on the boundary. Raw replace is similar to generalized power iteration for solving Rayleigh quotient problems. In our experiments, replace-style updates are fast and often strong on Burgers and Darcy Flow, whereas add-style updates are more stable for recurrent Navier–Stokes.

Solver-Integrated Adversarial Training

The training stage wraps the attack objectives above in an online attack-then-train loop. We start from a trained baseline neural operator. At step t , the current model is frozen during attack generation, and an attacked batch $x^{\text{adv}} = x + \delta_N$ is obtained by maximizing one of $\mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3$, or $\mathcal{L}_{\text{phys}}$ for N projected steps. This attack step searches for locally high-loss input functions where the current model has large model–solver discrepancy or large physics residual. The trainable model is then updated on the attacked batch, and the updated weights become the next current model.

This process can be summarized as a function-space min-max risk. Let $\mu_{\mathcal{X}}$ be the data distribution over the input function space $\mathcal{X}$, so that $x \sim \mu_{\mathcal{X}}$. The usual expected risk is $\mathcal{R}(\theta) = \mathbb{E}_{x \sim \mu_{\mathcal{X}}}[\mathcal{L}_\theta(x)]$. The adversarial attack moves the sample x toward a higher-loss region using a perturbation δ within the ε -ball. The adversarial training objectives are therefore

$$\min_{\theta} \mathcal{R}_{\text{robust}}(\theta) = \min_{\theta} \mathbb{E}_{x \sim \mu_{\mathcal{X}}} \left[\max_{\|\delta\|_{\mathcal{X},p} \leq \varepsilon} \mathcal{L}_\theta(x; \delta) \right].$$

In our experiments, we also include two hand-designed random perturbation training methods. These variants draw perturbations from a fixed random sampler with ε jitter that is independent of the model at the current training step. They differ only in target: *random solver* uses $y = \mathcal{G}(x + \delta)$, while *random clean* keeps $y = \mathcal{G}(x)$.

Experiments

Benchmark PDEs

We evaluate on the three canonical PDE operator-learning benchmarks used in the Fourier Neural Operator (FNO) paper (Li et al. 2021): 1D Burgers, 2D Darcy Flow, and 2D Navier–Stokes. Due to page limits, many experimental results have been moved to the supplement.

Solver-Integrated Adversarial Attacks

Except for Burgers at $\nu = 10^{-2}$, where we also include DeepONet, all other experiments use FNO. For Burgers and recurrent Navier–Stokes, we compare $\mathcal{L}_1, \mathcal{L}_2$, and $\mathcal{L}_3$; for Darcy Flow, we additionally compare $\mathcal{L}_{\text{phys}}$. In visualizations and auxiliary sweeps, we also compare $\mathcal{L}_3^{\text{sg}}, \mathcal{L}_{2,N}^{\text{dict}}$ with $N \in \{200, 2000, 20000\}$, and $\mathcal{L}_2^{\text{fixed}}$. Across these settings,

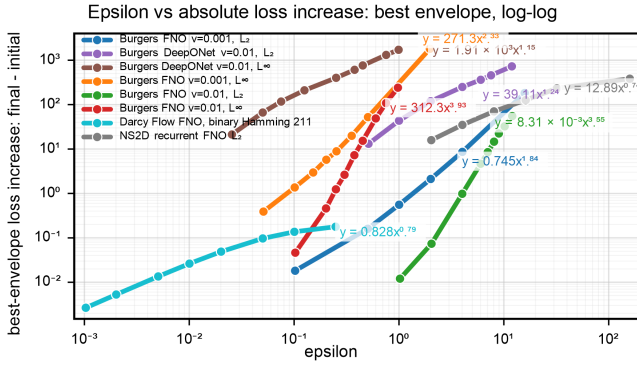


Figure 1: Power-law relationship between attack budget and loss increase

Table 1: **Attack Objective Comparison.** $\mathcal{L}_3$ is used as the true loss here, reported as the sample mean. Superscript * marks a one-sided test against the second-highest objective ($q \leq 0.05$).

Burgers (100 attack steps, $N = 50$ samples)						
Budget	Step size	$\mathcal{L}_1 \uparrow$	$\mathcal{L}_2 \uparrow$	$\mathcal{L}_3 \uparrow$	$\mathcal{L}_3$ win $\uparrow$	
$\varepsilon = 0.125$	0.0125	0.1027	0.1021	0.1306*	50/50	
$\varepsilon = 0.25$	0.025	0.1008	0.0999	0.1664*	50/50	
$\varepsilon = 0.5$	0.05	0.0982	0.0963	0.2793*	50/50	
$\varepsilon = 0.75$	0.075	0.0976	0.0934	0.4612*	50/50	
$\varepsilon = 1.0$	0.1	0.0988	0.0913	0.7262*	50/50	
Darcy Flow (100 attack steps, $N = 20$ samples)						
Budget	Step size	$\mathcal{L}_1 \uparrow$	$\mathcal{L}_2 \uparrow$	$\mathcal{L}_3 \uparrow$	$\mathcal{L}_{\text{phys}} \uparrow$	$\mathcal{L}_3$ win $\uparrow$
$K = 100$	5	0.0254	0.0256	0.0307*	0.0245	14/20
$K = 250$	5	0.0266	0.0270	0.0403*	0.0245	17/20
$K = 437$	5	0.0276	0.0284	0.0500*	0.0244	17/20
$K = 875$	5	0.0288	0.0309	0.0771*	0.0245	20/20
Navier–Stokes (50 attack steps, $N = 20$ samples)						
Budget	Step size	$\mathcal{L}_1 \uparrow$	$\mathcal{L}_2 \uparrow$	$\mathcal{L}_3 \uparrow$	$\mathcal{L}_3$ win $\uparrow$	
$\varepsilon = 8$	0.25	64.979	59.567	126.832*	20/20	
$\varepsilon = 16$	0.5	68.913	61.469	186.683*	20/20	
$\varepsilon = 32$	1	118.588	69.505	276.760*	20/20	
$\varepsilon = 64$	2	190.810	88.554	324.618*	18/20	

the attack-loss increase grows approximately as a power law in the attack budget, $\Delta \mathcal{L} \approx C\varepsilon^a$, as summarized in Figure 1.

The main result is simple: to make the model output differ strongly from the solver output, the attack should optimize $\mathcal{L}_3^{\text{full}}$ directly. As shown in Table 1, attacks based on $\mathcal{L}_1$, $\mathcal{L}_2$, $\mathcal{L}_{\text{phys}}$, $\mathcal{L}_2^{\text{fixed}}$, $\mathcal{L}_{2,N}^{\text{dict}}$, or $\mathcal{L}_3^{\text{sg}}$ increase the true $\mathcal{L}_3$ more slowly and reach a smaller final $\mathcal{L}_3$. Therefore, we need to use the solver both in the forward pass and in the backward pass through DG . See Figures 2 and 3 for attack visualizations.

Our second finding concerns frequency content. In Burgers and Navier–Stokes, perturbations generated by full $\mathcal{L}_3$ concentrate on higher-frequency modes than those from other objectives. The local SVD diagnostic quantifies the mechanism: for Burgers FNO at $\nu = 10^{-2}$, after removing the DC mode, the q_{50} frequency of the rank-1 right singular vector is 0.20% of the Fourier grid for both $D\mathcal{F}_\theta[x]$ and $DG[x]$,

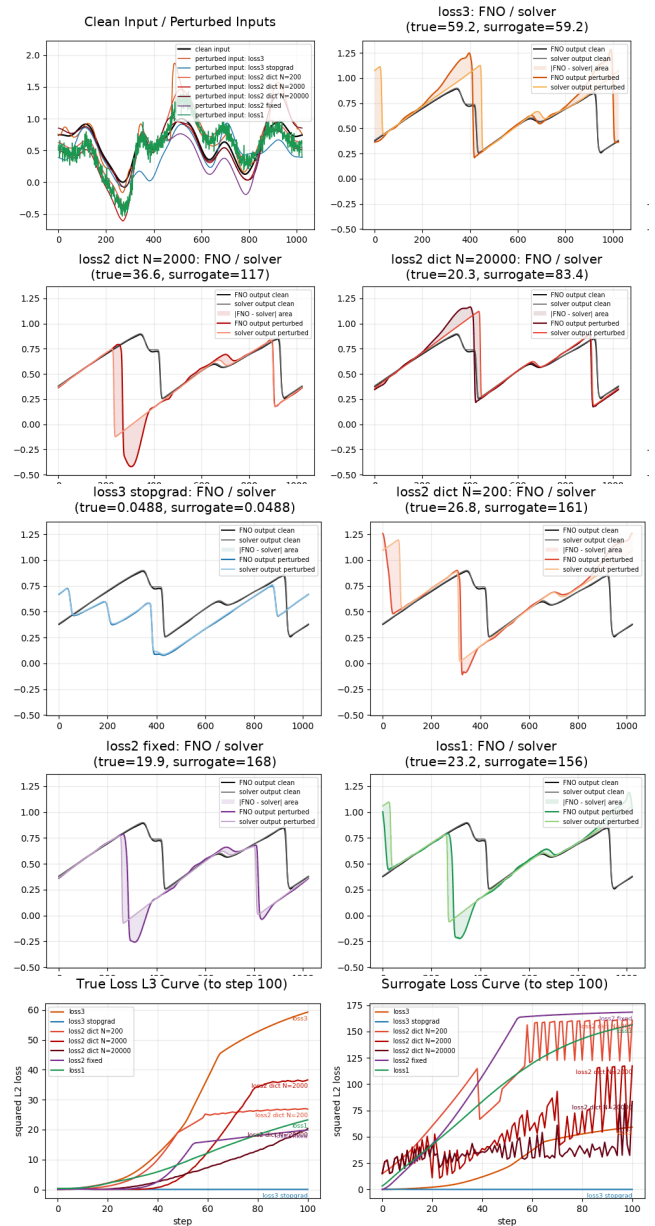


Figure 2: Burgers FNO L_2 PGD attack variants.

but 15.7% for $D\mathcal{E}_\theta[x]$. For ranks 2–4, the corresponding q_{50} values remain at most 0.39% for $D\mathcal{F}_\theta[x]$ and $DG[x]$, while $D\mathcal{E}_\theta[x]$ ranges from 45.4% to 69.3%. Thus, to maximize the model–solver difference, the perturbation must move along this sharper high-frequency direction.

Projected Attack Optimization Methods

The optimizer ablations give a surprising pattern: for Burgers and Darcy Flow, replace-style updates move to the perturbation boundary on the first replacement update and then attain high final true $\mathcal{L}_3$. In the saved Burgers diagnostic, steepest replace ends at $\mathcal{L}_3 = 7.069$, while steepest add ends at 6.339, a 10.3% gap relative to replace. In Darcy Flow, binary replace ends at 0.05285, while binary add ends at

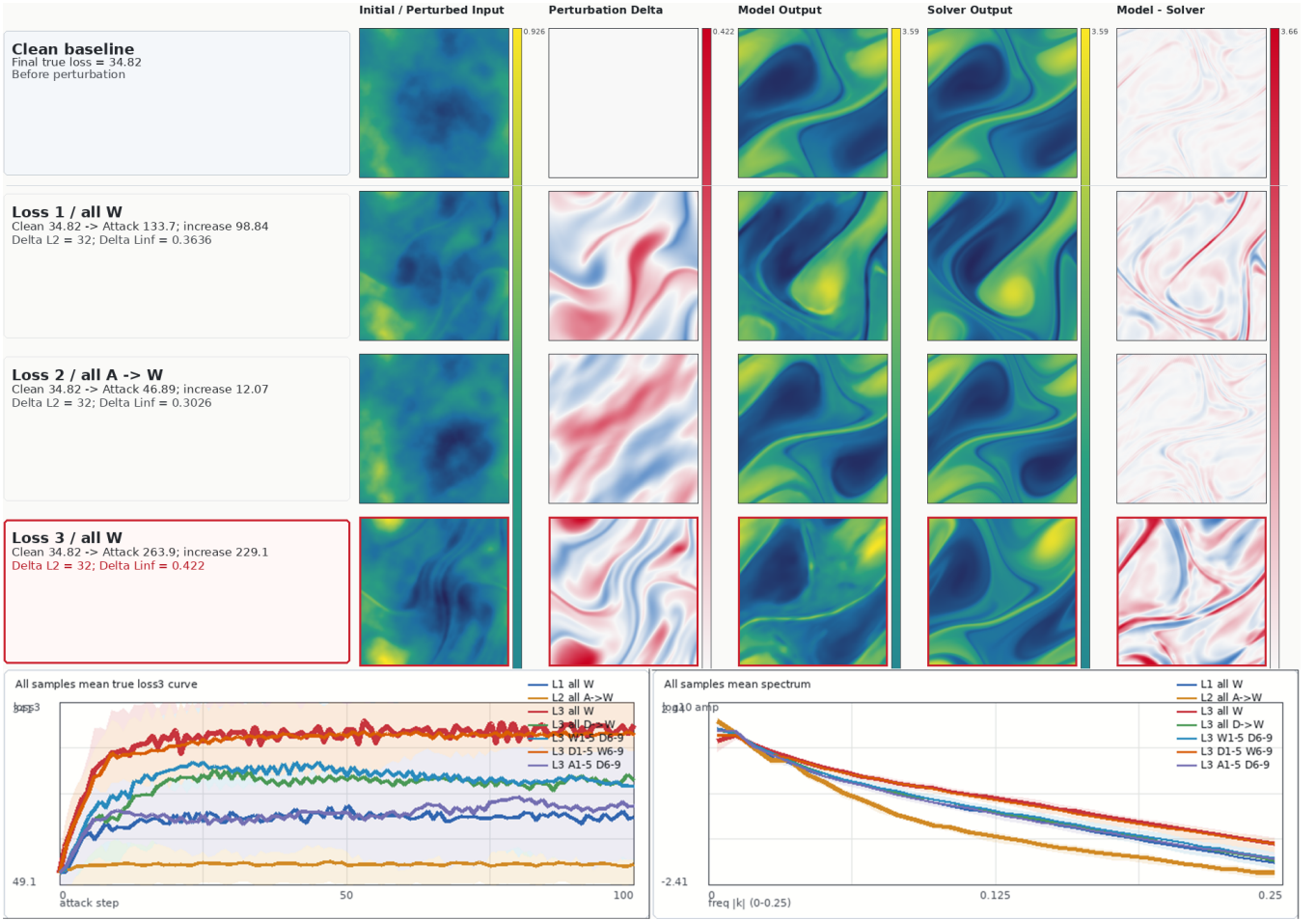


Figure 3: Loss 3 creates the largest model-solver discrepancy for 2D Navier–Stokes. Bottom: all-sample mean true-loss curve and perturbation spectrum.

0.04809, a 9.0% gap relative to replace. Mechanism diagnostics suggest two reasons: replace reaches the constraint boundary immediately instead of slowly moving outward, and the Burgers/Darcy loss landscapes contain broad connected high-loss regions, so a coarse boundary move can already enter such a region. Recurrent Navier–Stokes is different: add-style optimization reaches a much larger $\mathcal{L}_3$ than replace, indicating a narrower and sharper high-loss region that is hard to hit without more refined iterative optimization.

Adversarial Training

Because different training methods have different per-epoch time costs, we compare all models at a similar total wall-clock training time. We measure both generalization to shifted datasets and robustness under adversarial attack. These broad generalization datasets are generated by shifting the input distribution: we vary GRF kernel families, correlation lengths, and spectral-decay parameters, include non-GRF function families, and change amplitude ranges or thresholding transforms. For Burgers and Darcy Flow, we conduct detailed and complete experimental comparisons across various training methods.

Generalization. On Burgers (Figure 4), Loss 3 gives the lowest generalization relative L_2 on 48 of the 50 shifted datasets. Random solver is the second-best method and performs very well early in wall-clock time, but after about four hours it shows clear overfitting: its generalization loss starts to increase. We infer that the main reason is that its δ is manually prescribed in advance from a fixed statistical distribution, so it cannot use adversarial attack against the current model to generate perturbations along the model’s current weak directions. As shown in Figure 6, $\mathcal{L}_3$ adversarial training generates perturbations that clearly become higher-frequency as training time increases, while $\mathcal{L}_1$ and $\mathcal{L}_2$ training do not show this obvious high-frequency shift. Darcy Flow (Figure 5) shows the same pattern, with Loss 3 giving the lowest generalization relative L_2 on 47 of the 50 shifted datasets. For both Burgers and Darcy Flow, the $\mathcal{L}_3$ -trained model is not the best model on the original train/test datasets. This suggests a possible trade-off between in-distribution and out-of-distribution performance: a model that performs better on shifted datasets can perform worse on the original train/test distribution.

For 2D Navier–Stokes, we do not compare different training methods; we only run $\mathcal{L}_3$ adversarial training, with six

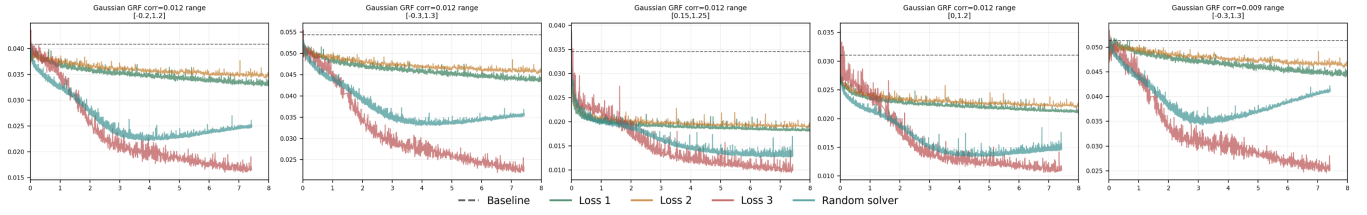


Figure 4: Burgers selected shifted-dataset generalization loss curves of all training methods. (Random clean is omitted here because it performs poorly and appears outside the plotted loss scale.)

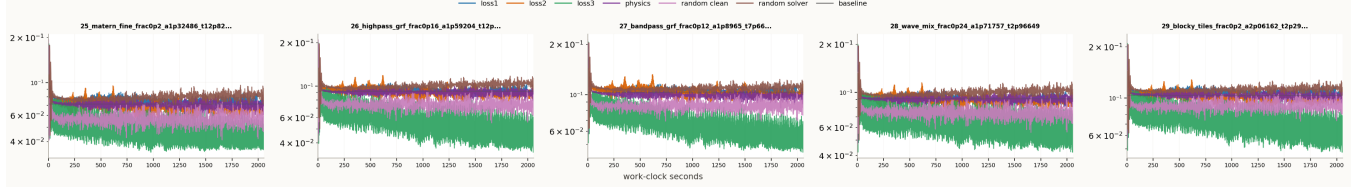


Figure 5: Darcy Flow selected shifted-dataset generalization loss curves of all training methods.

Table 2: **Adversarial trained models robustness.** Values are sample means. Attack loss uses 50 shifted datasets with 50 samples in each dataset. The Jacobian-error metric uses 25 selected data points. Superscript * marks a one-sided test against the second-lowest value ($q \leq 0.05$).

Burgers

Metric	Base	$\mathcal{L}_1$	$\mathcal{L}_2$	$\mathcal{L}_3$	Rand. clean	Rand. solver
Attack $\Delta L \downarrow$	0.0092	0.0058	0.0056	0.0031*	0.0428	0.0080
J-error $\downarrow$	0.4630	0.3159	0.3433	0.1119*	4.7850	0.3712

Darcy Flow

Metric	Base	$\mathcal{L}_1$	$\mathcal{L}_2$	$\mathcal{L}_3$
Attack $\Delta L \downarrow$	9.681e-6	3.927e-6	3.565e-6	1.699e-6*
J-error $\downarrow$	9.293e-5	9.922e-5	9.225e-5	6.157e-5*

Metric	$\mathcal{L}_{\text{phys}}$	Rand. clean	Rand. solver
Attack $\Delta L \downarrow$	3.974e-6	3.691e-6	4.824e-6
J-error $\downarrow$	1.034e-4	9.112e-5	1.236e-4

rounds in total. The main reason is that Navier–Stokes adversarial training requires very long wall-clock time and very large GPU memory, and we do not have sufficient computational resources. After training, 364 out of 402 datasets show lower MSE.

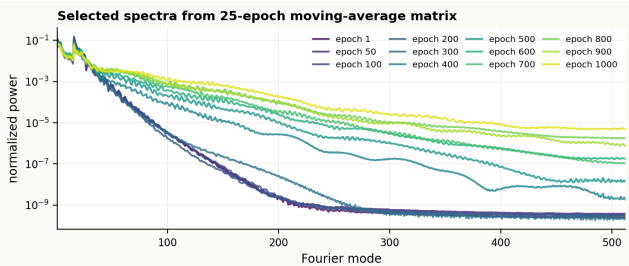


Figure 6: Increasingly higher-frequency perturbation in Burgers Loss-3 adversarial training.

Robustness. Table 2 reports finite-budget attack-loss increase and the Jacobian-error norm $\|J_{\mathcal{E}}^T e_{\theta}\|_2$. Loss 3 is strongest on both Burgers and Darcy Flow for attack loss and Jacobian-error robustness. We do not report the Jacobian spectral norm because it correlates less with finite-budget attack-loss increase and perturbation directions, especially at small ε .

Conclusion

This paper studies the generalizability and robustness of neural operators from a solver-integrated perspective. We define both quantities through model–solver discrepancy under distribution shift and adversarial perturbation, show why model-only robustness indicators are insufficient for PDE operator learning. We then propose a solver-integrated adversarial attack whose forward loss and backward gradient both use the PDE solver, so the attack directly maximizes the relevant model–solver error. From this attack, we derive an adversarial-training procedure that generates solver-relabelled adversarial batches online. Experiments on Burgers, Darcy Flow, and Navier–Stokes settings show that this solver-integrated adversarial training clearly improves both generalizability and robustness over non-solver-integrated objectives, without manually selecting special shifted training datasets. More broadly, the framework is not limited to operator learning: any neural-network system with access to a ground-truth oracle can adopt this formulation for adversarial attacks and adversarial training.

The main limitation is computational cost. Solver-integrated attacks and training require expensive solver forward passes and gradients through the solver. For time-dependent PDEs, the solver backward pass must keep the intermediate states and stage variables from many time steps in GPU memory so that gradients can be propagated back through the entire rollout. This creates a large memory footprint and a long wall-clock time cost, especially for Navier–Stokes. A future direction is therefore to reduce the solver time and memory cost incurred by adversarial training.

References

- Adesoji, A. D.; and Chen, P.-Y. 2022. Evaluating the Adversarial Robustness for Fourier Neural Operators. In *ICLR 2022 Workshop on Socially Responsible Machine Learning*.
- Andriushchenko, M.; Croce, F.; Flammarion, N.; and Hein, M. 2020. Square Attack: A Query-Efficient Black-Box Adversarial Attack via Random Search. In *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science, 484–501. Springer.
- Bousquet, O.; and Elisseeff, A. 2002. Stability and Generalization. *Journal of Machine Learning Research*, 2: 499–526.
- Brendel, W.; Rauber, J.; and Bethge, M. 2018. Decision-Based Adversarial Attacks: Reliable Attacks Against Black-Box Machine Learning Models. In *International Conference on Learning Representations*.
- Brown, T. B.; Mané, D.; Roy, A.; Abadi, M.; and Gilmer, J. 2017. Adversarial Patch. In *NIPS 2017 Workshop on Machine Learning and Computer Security*.
- Carlini, N.; and Wagner, D. 2017. Towards Evaluating the Robustness of Neural Networks. In *IEEE Symposium on Security and Privacy*, 39–57.
- Chen, J.; Jordan, M. I.; and Wainwright, M. J. 2020. Hop-SkipJumpAttack: A Query-Efficient Decision-Based Attack. In *IEEE Symposium on Security and Privacy*, 1277–1294.
- Cissé, M.; Bojanowski, P.; Grave, E.; Dauphin, Y.; and Usunier, N. 2017. Parseval Networks: Improving Robustness to Adversarial Examples. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, 854–863. PMLR.
- Croce, F.; and Hein, M. 2020. Reliable Evaluation of Adversarial Robustness with an Ensemble of Diverse Parameter-Free Attacks. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, 2206–2216. PMLR.
- Enyeart, D.; and Lin, G. 2024. Adversarial Autoencoders in Operator Learning. *arXiv preprint arXiv:2412.07811*.
- Goodfellow, I. J.; Shlens, J.; and Szegedy, C. 2015. Explaining and Harnessing Adversarial Examples. In *International Conference on Learning Representations*.
- Huang, C.; Ma, C.; Wang, K.; and Xiang, Y. 2026. StableP-DENet: Enhancing Stability of Operator Learning for Solving Differential Equations. *arXiv preprint arXiv:2601.06472*.
- Li, Z.; Kovachki, N.; Aizzadenesheli, K.; Liu, B.; Bhattacharya, K.; Stuart, A.; and Anandkumar, A. 2021. Fourier Neural Operator for Parametric Partial Differential Equations. In *International Conference on Learning Representations*.
- Liu, L.; Park, Y.; Hoang, T. N.; Hasson, H.; and Huan, J. 2023. Robust Multivariate Time-Series Forecasting: Adversarial Attacks and Defense Mechanisms. In *International Conference on Learning Representations*.
- Lu, L.; Jin, P.; Pang, G.; Zhang, Z.; and Karniadakis, G. E. 2021. Learning Nonlinear Operators via DeepONet Based on the Universal Approximation Theorem of Operators. *Nature Machine Intelligence*, 3(3): 218–229.
- Madry, A.; Makelov, A.; Schmidt, L.; Tsipras, D.; and Vladu, A. 2018. Towards Deep Learning Models Resistant to Adversarial Attacks. In *International Conference on Learning Representations*.
- Mode, G. R.; and Hoque, K. A. 2020. Adversarial Examples in Deep Learning for Multivariate Time Series Regression. In *IEEE Applied Imagery Pattern Recognition Workshop*, 1–10.
- Moosavi-Dezfooli, S.-M.; Fawzi, A.; and Frossard, P. 2016. DeepFool: A Simple and Accurate Method to Fool Deep Neural Networks. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2574–2582.
- Nguyen, A. T.; and Raff, E. 2019. Adversarial Attacks, Regression, and Numerical Stability Regularization. In *AAAI-19 Workshop on Engineering Dependable and Secure Machine Learning Systems*.
- Oommen, V.; Khodakarami, S.; Bora, A.; Wang, Z.; and Karniadakis, G. E. 2025. Learning Turbulent Flows with Generative Models: Super-resolution, Forecasting, and Sparse Flow Reconstruction. *arXiv preprint arXiv:2509.08752*.
- Ouyang, W.; Zhu, M.; Xiong, W.; Liu, S.-W.; and Lu, L. 2025. RAMS: Residual-Based Adversarial-Gradient Moving Sample Method for Scientific Machine Learning in Solving Partial Differential Equations. *arXiv preprint arXiv:2509.01234*.
- Papernot, N.; McDaniel, P.; Jha, S.; Fredrikson, M.; Celik, Z. B.; and Swami, A. 2016. The Limitations of Deep Learning in Adversarial Settings. In *IEEE European Symposium on Security and Privacy*, 372–387.
- Rahman, M. A.; Florez, M. A.; Anandkumar, A.; Ross, Z. E.; and Aizzadenesheli, K. 2022. Generative Adversarial Neural Operators. *Transactions on Machine Learning Research*.
- Ribeiro, A. H.; Zachariah, D.; Bach, F.; and Schön, T. B. 2023. Regularization Properties of Adversarially-Trained Linear Regression. In *Advances in Neural Information Processing Systems*, volume 36, 23658–23670.
- Roy, S.; Chakraborty, S.; and Alam, S. B. 2026. Beyond Uniform Sampling: Synergistic Active Learning and Input Denoising for Robust Neural Operators. *arXiv preprint arXiv:2604.13316*.
- Roy, S.; Kobayashi, K.; Chakraborty, S.; Rizwan-uddin; and Alam, S. B. 2026. Adversarial Vulnerabilities in Neural Operator Digital Twins: Gradient-Free Attacks on Nuclear Thermal-Hydraulic Surrogates. *arXiv preprint arXiv:2603.22525*.
- Shi, S.; Li, Y.; Guo, Z.; Wu, B.; and Zhao, Y. 2026. PINNs Failure Region Localization and Refinement through White-box Adversarial Attack. *Neurocomputing*, 664: 132055.
- Su, J.; Vargas, D. V.; and Sakurai, K. 2019. One Pixel Attack for Fooling Deep Neural Networks. *IEEE Transactions on Evolutionary Computation*, 23(5): 828–841.
- Tong, L.; Yu, S.; Alfeld, S.; and Vorobeychik, Y. 2018. Adversarial Regression with Multiple Learners. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, 4946–4954. PMLR.
- Xu, H.; and Mannor, S. 2012. Robustness and Generalization. *Machine Learning*, 86(3): 391–423.